



Chapter – 5

Node.JS





Node.js is a cross-platform environment and library for running JavaScript applications which is used to create networking and server-side applications.

5.1 Concept, Working and Features

Node.js is a cross-platform runtime environment and library for running JavaScript applications outside the browser. It is used for creating server-side and networking web applications. It is open source and free to use. It can be downloaded from this link <https://nodejs.org/en/>

Many of the basic modules of Node.js are written in JavaScript. Node.js is mostly used to run real-time server applications.

The definition given by its official documentation is as follows:

“Node.js is a platform built on Chrome's JavaScript runtime for easily building fast and scalable network applications. Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient, perfect for data-intensive real-time applications that run across distributed devices.”

Node.js also provides a rich library of various JavaScript modules to simplify the development of web applications.

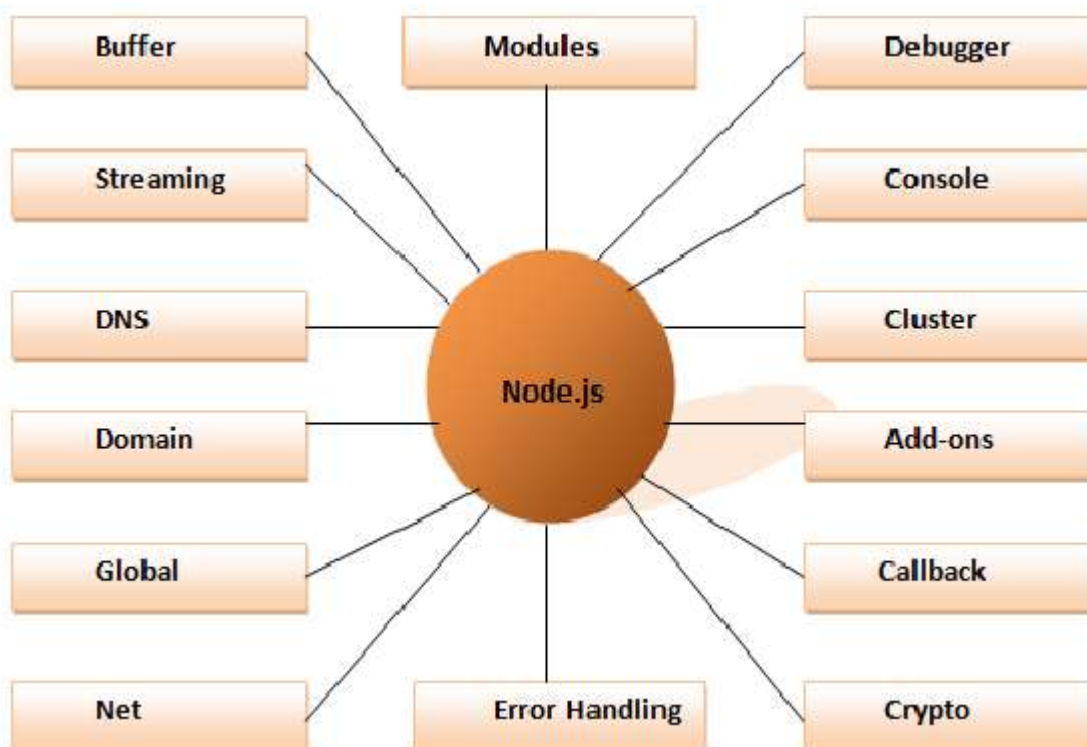


Fig:5.1 Components of Node JS



Features of Node.JS

Following is a list of some important features of Node.js that makes it the first choice of software architects.

1. **Extremely fast:** Node.js is built on Google Chrome's V8 JavaScript Engine, so its library is very fast in code execution.
2. **I/O is Asynchronous and Event Driven:** All APIs of Node.js library are asynchronous i.e. non-blocking. So a Node.js based server never waits for an API to return data. The server moves to the next API after calling it and a notification mechanism of Events of Node.js helps the server to get a response from the previous API call. It is also a reason that it is very fast.
3. **Single threaded:** Node.js follows a single threaded model with event looping.
4. **Highly Scalable:** Node.js is highly scalable because event mechanism helps the server to respond in a non-blocking way.
5. **No buffering:** Node.js cuts down the overall processing time while uploading audio and video files. Node.js applications never buffer any data. These applications simply output the data in chunks.
6. **Open source:** Node.js has an open source community which has produced many excellent modules to add additional capabilities to Node.js applications.
7. **License:** Node.js is released under the MIT license.

5.1.1 Downloading Node.JS

To install and setup an environment for Node.js, you need the following two software available on your computer:

1. Text Editor.
2. Node.js Binary installable

You can download the latest version of Node.js installable archive file from <https://nodejs.org/en/>. Details of installation is mentioned in the next topic.

5.2 Setting up Node.JS server (HTTP Server)

5.2.1 Installing On Windows

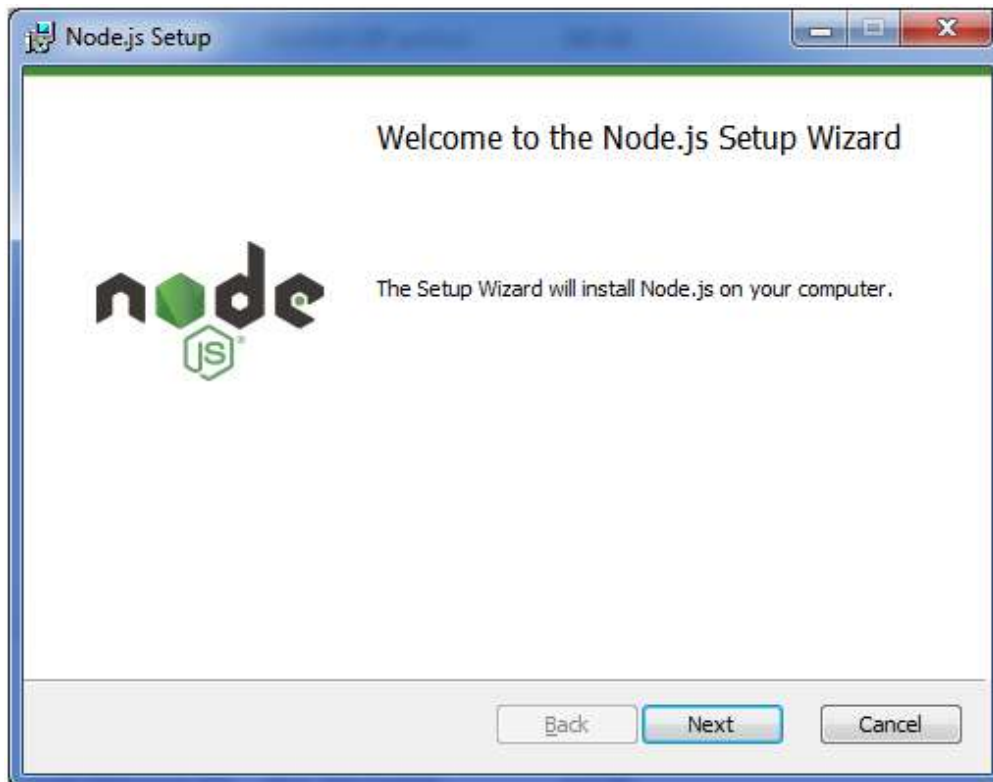
There are various installers available on <https://nodejs.org/en/>. The installation files are available for Windows, Linux, Solaris, MacOS.



Download the installer for windows by clicking on LTS or Current version button. Here, we will install the latest version LTS for windows that has long time support. However, you can also install the Current version which will have the latest features.

After you download the MSI, double-click on it to start the installation as shown below.



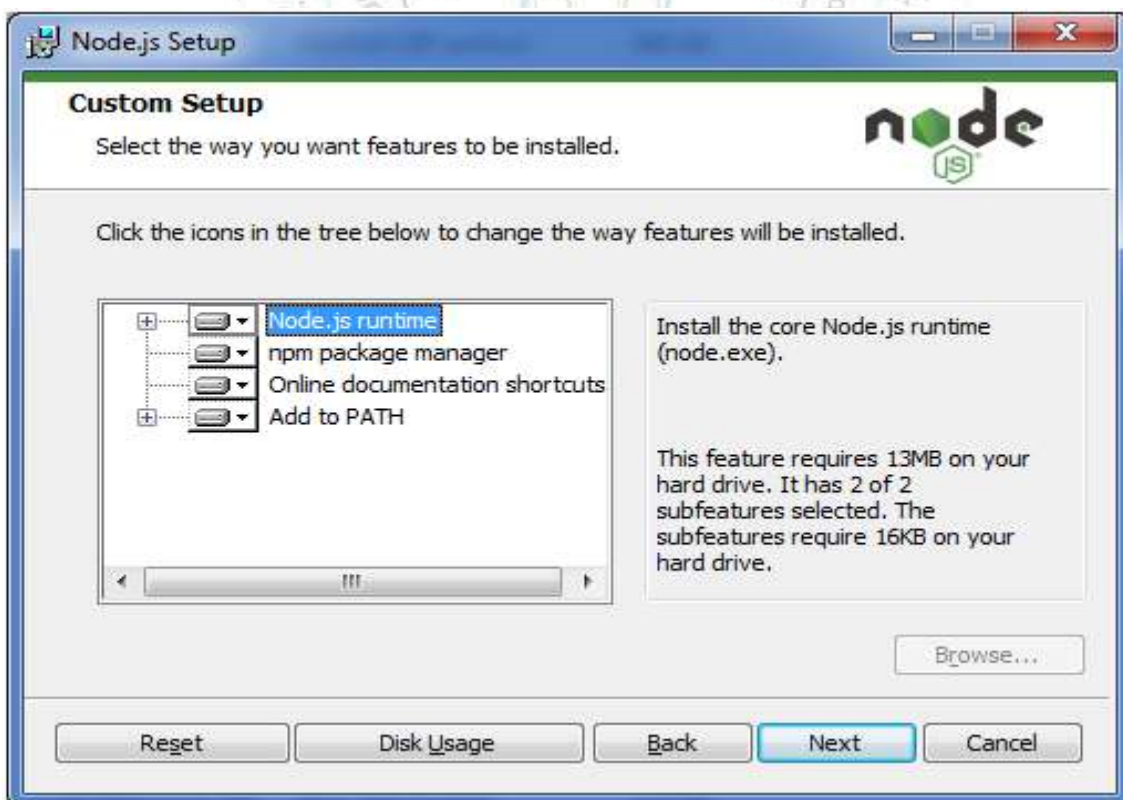
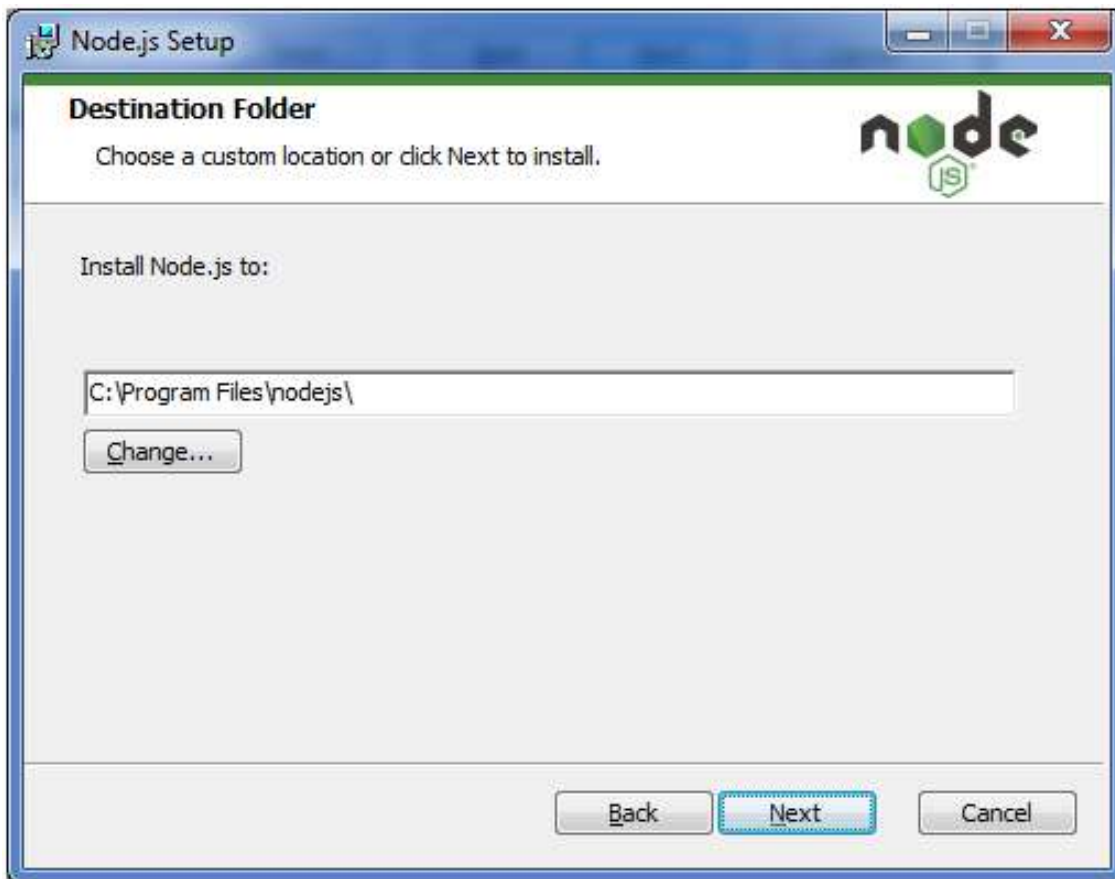


Accept the terms of license agreement.



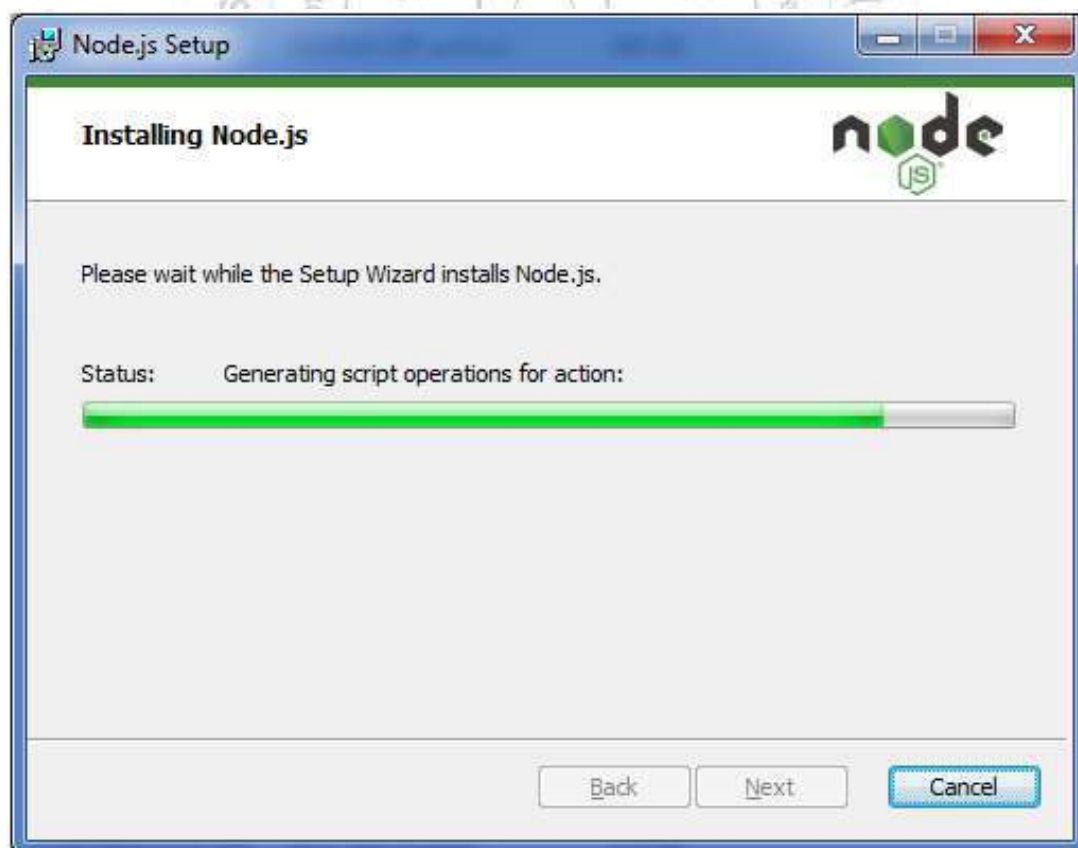


Choose the location where you want to install.





Ready to install:





Verify Installation

Once you install Node.js on your computer, you can verify it by opening the command prompt and typing `node -v`. If Node.js is installed successfully then it will display the version of the Node.js installed on your machine, as shown below.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19043.1237]
(c) Microsoft Corporation. All rights reserved.

C:\Users\91963>node -v
v14.18.0

C:\Users\91963>
```



5.2.2 Components

A Node.js application consists of the following three important components –

1. **Import required modules:** The "require" directive is used to load a Node.js module.
2. **Create server:** You have to establish a server which will listen to client's request similar to Apache HTTP Server.
3. **Read request and return response:** Server created in the second step will read HTTP request made by client which can be a browser or console and return the response.

We will see them in details below:

5.2.2.1 Required modules, Create Server (`http.createServer()`)

Require Modules: Method `require()` is used to consume modules. It allows you to include modules in your app. You can add built-in core Node.js modules, community-based modules (node_modules), and local modules too.

Node.js follows the CommonJS module system, and the built-in `require` function is the easiest way to include modules that exist in separate files. The basic functionality of `require` is that it reads a JavaScript file, executes the file, and then proceeds to return the exports object.

```
var module = require('module_name');
```

As per above syntax, specify the module name in the `require()` function. The `require()` function will return an object, function, property or any other JavaScript type, depending on what the specified module returns.

We use the `require` directive to load the `http` module and store the returned HTTP instance into an `http` variable as follows – **block1**

```
var http = require("http");
```

In the above example, `require()` function returns an object because `http` module returns its functionality as an object, you can then use its properties and methods using dot notation e.g. `http.createServer()`.

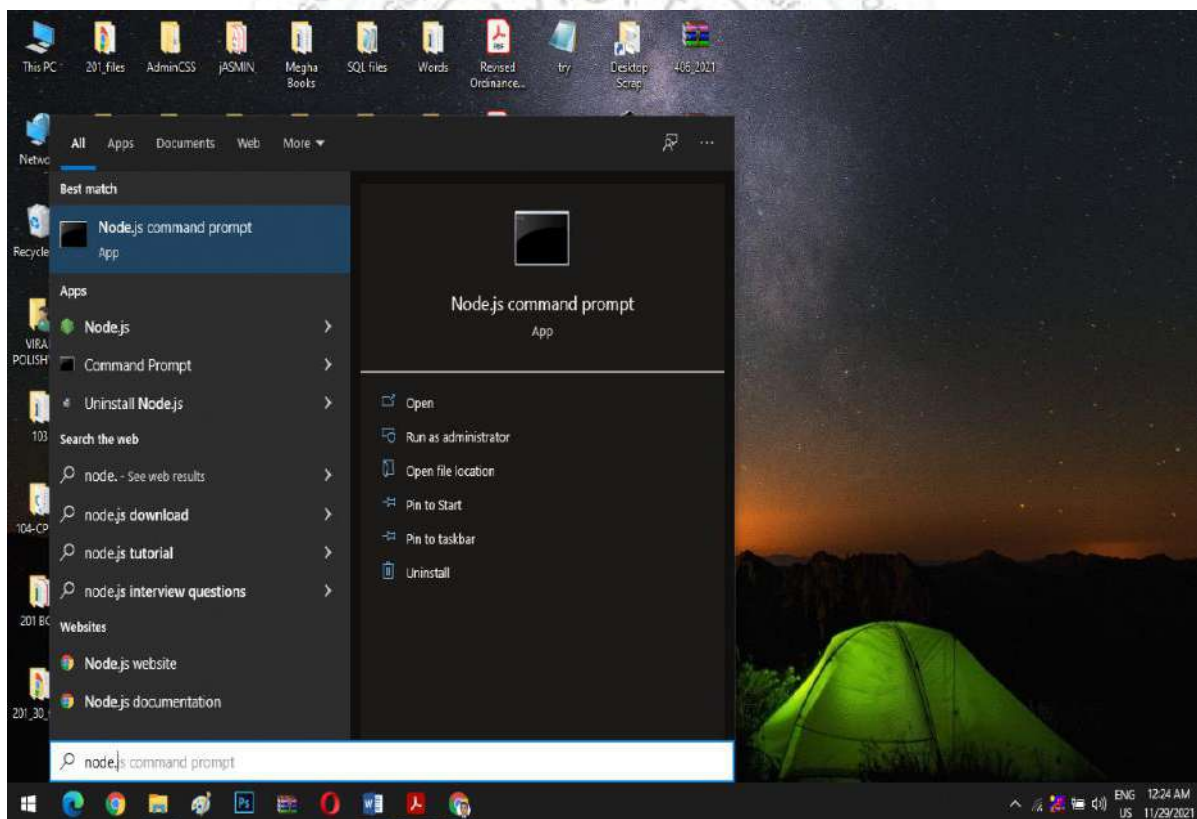
Create server: In the second component, you have to use created `http` instance and call `http.createServer()` method to create server instance and then bind it at port 8081 using `listen` method associated with server instance. Pass it a function with request and response parameters and write the sample implementation to return "Hello World". Check the example below: **block2**



```
http.createServer(function (request, response) {  
  // Send the HTTP header  
  // HTTP Status: 200 : OK  
  // Content Type: text/plain  
  response.writeHead(200, {'Content-Type': 'text/plain'});  
  // Send the response body as "Hello World"  
  response.end('Hello World\n');  
}).listen(8081);  
// Console will print the message  
console.log('Server running at http://127.0.0.1:8081/');
```

5.2.2.2 Request and Response

Now combine the block 1 and block 2 and save the file as **check.js**; while executing the above it will just create an HTTP server which listens to the port mentioned as 8081 on the local machine.



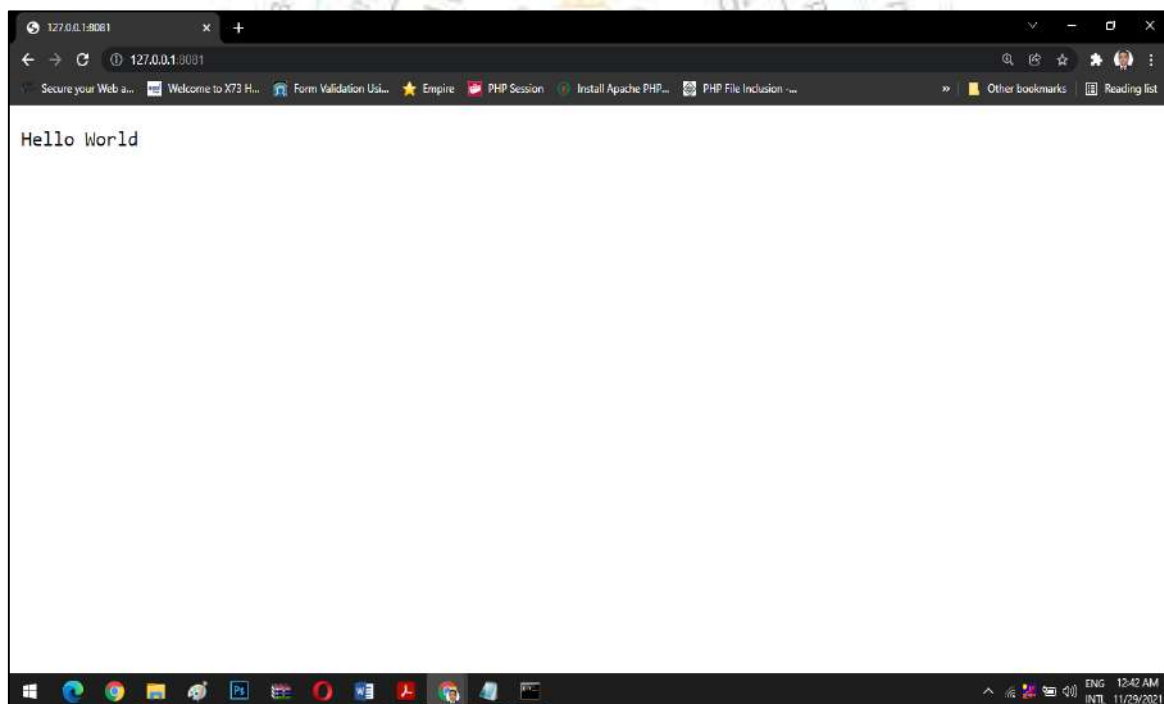
For starting the server, we need to start the command prompt as follows:



Click the start menu and type node.js command prompt. Once the command prompt is open just move the location where the check.js file is stored.

Once you get the message like server is running at , We can just open

```
Node.js command prompt - node check.js
C:\Users\Viral Polishwala>cd OneDrive
C:\Users\Viral Polishwala\OneDrive>cd Desktop
C:\Users\Viral Polishwala\OneDrive\Desktop>cd nodejs
C:\Users\Viral Polishwala\OneDrive\Desktop\nodejs>node check.js
Server running at http://127.0.0.1:8081/
```



any browser and check for the <http://127.0.0.1:8081/>

Now, if you make any changes in the " check.js" file, you need to again run the "node check.js" command.

This is how the http server is created and request which are sent over https server are responded from the port 8081.



5.3 Built-In Modules

In Node.js, Modules are the blocks of encapsulated code that communicates with an external application on the basis of their related functionality. Modules can be a single file or a collection of multiples files/folders. The reason programmers are heavily reliant on modules is because of their re-usability as well as the ability to break down a complex piece of code into manageable chunks.

Node.js has many built-in modules that are part of the platform and comes with Node.js installation. Node.js Core Modules comes with its installation by default. You can use them as per application requirements. These modules can be loaded into the program by using the **require** function.

5.3.1 require () function

The `require ()` function will return a JavaScript type depending on what the particular module returns.

```
var module = require('module_name');
```

The following example demonstrates how to use the Node.js Http module to create a web server.

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write('Welcome to this page!');
  res.end();
}).listen(3000);
```

In the above example, the `require()` function returns an object because the Http module returns its functionality as an object. The function `http.createServer()` method will be executed when someone tries to access the computer on port 3000. The `res.writeHead()` method is the status code where 200 means it is OK, while the second argument is an object containing the response headers.

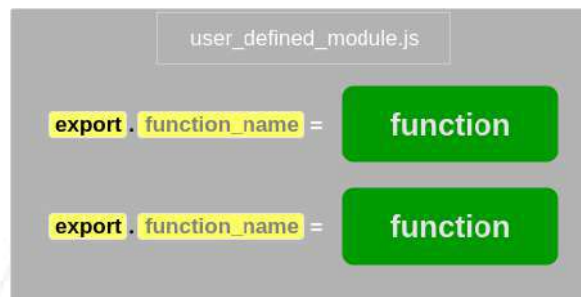
Module	Description
OS Module	Provides basic operating-system related utility functions. <code>var os = require("os")</code>
Path Module	Provides utilities for handling and transforming file paths. <code>var path = require("path")</code>
Net Module	Provides both servers and clients as streams. Acts as a network wrapper. <code>var net = require("net")</code>
DNS Module	Provides functions to do actual DNS lookup as well as to use underlying operating system name resolution functionalities. <code>var dns = require("dns")</code>



Domain Module	Provides ways to handle multiple different I/O operations as a single group. var domain = require("domain")
---------------	--

5.3.2 User Defined Modules: Create and include

Modules are the collection of JavaScript codes in a separate logical file that can be used in external applications on the basis of their related functionality. Modules are popular as they are easy to use and are reusable. Sometimes it is required that, when you are implementing a Node.js application for a use case, you might want to keep your business logic separately. In such cases you create a Node.js module with all the required functions in it.



Create Modules (exports)

The module.exports is a special object which is included in every JavaScript file in the Node.js application by default. The module is a variable that represents the current module, and exports is an object that will be exposed as a module. So, whatever you assign to module.exports will be exposed as a module.

To create a module in Node.js, the exports keyword is used. This keyword tells Node.js that the function can be used outside the module. A Node.js Module is a .js file with one or more functions.

The syntax to define a function in Node.js module is

```
exports.<function_name> = function (argument_1, argument_2, .. argument_N)
{
  /** function body */
};
```

- **exports** – is a keyword which tells Node.js that the function is available outside the module.
- **function_name** – is the function name using which we can access this function in other programs.

As mentioned above, exports is an object. So it exposes whatever you assigned to it as a module. For example, if you assign a string literal then it will expose that string literal as a module.



The following example exposes simple string message as a module in Message.js.

```
module.exports = 'Hello world';
```

Include Modules (require)

Node.js follows the CommonJS module system, and the built-in require function is the easiest way to include modules that exist in separate files. The basic functionality of require is that it reads a JavaScript file, executes the file, and then proceeds to return the exports object. An example module:

Now, import this message module and use it as shown below.

```
var msg = require('./Messages.js');  
console.log(msg);
```

Run the above example and see the result, as shown below.

```
C:\> node app.js  
Hello World
```

Export Object

The exports is an object. So, you can attach properties or methods to it. The following example exposes an object with a string property in Message.js file.

```
exports.SimpleMessage = 'Hello world';  
//or  
module.exports.SimpleMessage = 'Hello world';
```

In the above example, we have attached a property SimpleMessage to the exports object. Now, import and use this module, as shown below: app.js

```
var msg = require('./Messages.js');  
console.log(msg.SimpleMessage);
```

In the above example, the require() function will return an object { SimpleMessage : 'Hello World' } and assign it to the msg variable. So, now you can use msg.SimpleMessage.

Run the above example by writing node app.js in the command prompt and see the output as shown below.

```
C:\> node app.js  
Hello World
```

In the same way as above, you can expose an object with function. The following example exposes an object with the log function as a module: Log.js



```
module.exports.log = function (msg) {  
  console.log(msg); };
```

The above module will expose an object- { log : function(msg){ console.log(msg); } } . Use the above module as shown below: app.js

```
var msg = require('./Log.js');  
msg.log('Hello World');
```

Run and see the output in command prompt as shown below.

```
C:\> node app.js  
Hello World
```

You can also attach an object to module.exports, as shown below: data.js

```
module.exports = {  
  firstName: 'James',  
  lastName: 'Bond'  
}
```

app.js

```
var person = require('./data.js');  
console.log(person.firstName + ' ' + person.lastName);
```

Run the above example and see the result, as shown below.

```
C:\> node app.js  
James Bond
```

5.3.3 HTTP module

To make HTTP requests in Node.js, there is a built-in module HTTP in Node.js to transfer data over the HTTP. To use the HTTP server in node, we need to require the HTTP module. The HTTP module creates an HTTP server that listens to server ports and gives a response back to the client. The HTTP core module is a key module to Node.js networking. It is designed to support many features of the HTTP protocol.

We can create a HTTP server with the help of **http.createServer()** method. (Ex:testing.js)

```
var http = require('http');           // Create a server  
http.createServer((request, response)=>{           // Sends a chunk of the response body  
  response.write('Hello World!');  
  // Signals the server that all of the response headers and body have been sent  
  response.end();  
})  
.listen(3000);           // Server listening on port 3000
```



Above code will start the webserver by running the command **node testing.js**

```
var http = require('http');
var options = {
  host: 'www.amrolicollege.org',
  path: '/sports2122',
  method: 'GET'
};
// Making a get request to 'www.amrolicollege.org'
http.request(options, (response) => {      // Printing the statusCode
  console.log( STATUS: ${response.statusCode} );
}).end();
```

Above example will display the status code of the request made on server.

The HTTP module provides some properties and methods, and some classes.

http.METHODS

This property lists all the HTTP methods supported:

```
> require('http').METHODS
[ 'ACL', 'BIND', 'CHECKOUT', 'CONNECT', 'COPY', 'DELETE', 'GET', 'HEAD', 'LINK', 'LOCK', 'M-SEARCH',
'MERGE', 'MKACTIVITY', 'MKCALENDAR', 'MKCOL', 'MOVE', 'NOTIFY', 'OPTIONS', 'PATCH', 'POST',
'PROPFIND', 'PROPPATCH', 'PURGE', 'PUT', 'REBIND', 'REPORT', 'SEARCH', 'SUBSCRIBE', 'TRACE',
'UNBIND', 'UNLINK', 'UNLOCK', 'UNSUBSCRIBE']
```

http.STATUS_CODES

This property lists all the HTTP status codes and their description:

```
<img width="599" height="706" src="">> require('http').STATUS_CODES
{
  '100': 'Continue',
  '101': 'Switching Protocols',
  '102': 'Processing',
  '103': 'Early Hints',
  '200': 'OK',
  '201': 'Created',
  '202': 'Accepted',
  '203': 'Non-Authoritative Information',
  '204': 'No Content',
  '205': 'Reset Content',
  '206': 'Partial Content',
  '207': 'Multi-Status',
  '208': 'Already Reported',
  '226': 'IM Used',
  '300': 'Multiple Choices',
  '301': 'Moved Permanently',
  '302': 'Found',
  '303': 'See Other',
  '304': 'Not Modified',
  '305': 'Use Proxy',
  '307': 'Temporary Redirect',
```



```
'308': 'Permanent Redirect',
'400': 'Bad Request',
'401': 'Unauthorized',
'402': 'Payment Required',
'403': 'Forbidden',
'404': 'Not Found',
'405': 'Method Not Allowed',
'406': 'Not Acceptable',
'407': 'Proxy Authentication Required',
'408': 'Request Timeout',
'409': 'Conflict',
<img width="599" height="658" src=""> '410': 'Gone',
'411': 'Length Required',
'412': 'Precondition Failed',
'413': 'Payload Too Large',
'414': 'URI Too Long',
'415': 'Unsupported Media Type',
'416': 'Range Not Satisfiable',
'417': 'Expectation Failed',
'418': "I'm a Teapot",
'421': 'Misdirected Request',
'422': 'Unprocessable Entity',
'423': 'Locked',
'424': 'Failed Dependency',
'425': 'Too Early',
'426': 'Upgrade Required',
'428': 'Precondition Required',
'429': 'Too Many Requests',
'431': 'Request Header Fields Too Large',
'451': 'Unavailable For Legal Reasons',
'500': 'Internal Server Error',
'501': 'Not Implemented',
'502': 'Bad Gateway',
'503': 'Service Unavailable',
'504': 'Gateway Timeout',
'505': 'HTTP Version Not Supported',
'506': 'Variant Also Negotiates',
'507': 'Insufficient Storage',
'508': 'Loop Detected',
'509': 'Bandwidth Limit Exceeded',
'510': 'Not Extended',
'511': 'Network Authentication Required'
}
```

http.globalAgent

The `http.globalAgent` is a global object of the `http.Agent` class, which is utilized for all the HTTP client requests by default. It is used to control connections persistence, and reuse for HTTP clients. Moreover, it is a significant constituent in Node.js HTTP networking.

http methods are described in the next topic.

5.4 Node.JS as Web Server

When you view a webpage in your browser, you are making a request to another computer on the internet, which then provides you the webpage as a response. That



computer you are talking to via the internet is a *web server*. A web server receives HTTP requests from a client, like your browser, and provides an HTTP response, like an HTML page or JSON from an API. To access web pages of any web application, you need a web server. The web server will handle all the http requests for the web application e.g IIS is a web server for ASP.NET web applications and Apache is a web server for PHP or Java web applications.

A lot of software is involved for a server to return a webpage. This software generally falls into two categories: frontend and backend. *Front-end code* is concerned with how the content is presented, such as the color of a navigation bar and the text styling. *Back-end code* is concerned with how data is exchanged, processed, and stored. Code that handles network requests from your browser or communicates with the database is primarily managed by back-end code.

Node.js allows developers to use JavaScript to write back-end code, even though traditionally it was used in the browser to write front-end code. Having both the frontend and backend together like this reduces the effort it takes to make a web server, which is a major reason why Node.js is a popular choice for writing back-end code.

There are a variety of modules such as the “http” and “request” module, which helps in processing server related requests in the webserver space. We will have a look at how we can create a basic web server application using Node js.

Node.js provides capabilities to create your own web server which will handle HTTP requests asynchronously. You can use IIS or Apache to run Node.js web application but it is recommended to use Node.js web server.

5.4.1 **createserver (), writehead () method**

Node.js has a built-in module called HTTP, which allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP). The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client. Refer the code below to understand the concept; filename:helloserver.js

```
var http=require('http')
var server=http.createServer((function(request,response)
{
    response.writeHead(200,
    {"Content-Type" : "text/plain"});
    response.end("Hello World\n");
});
server.listen(8081);
```



1. The basic functionality of the require function is that it reads a JavaScript file, executes the file, and then proceeds to return the exports object. So in our case, since we want to use the functionality of the http module, we use the require function to get the desired functions from the http module so that it can be used in our application.
2. In this line of code, we are creating a server application which is based on a simple function. This function is called whenever a request is made to our server application. The request object can be used to get information about the current HTTP request e.g., url, request header, and data. The response object can be used to send a response for a current HTTP request.
3. When a request is received, we are saying to send a response with a header type of '200.' This number is the normal response which is sent in an http header when a successful response is sent to the client.
4. In the response itself, we are sending the string 'Hello World.'
5. We are then using the server.listen function to make our server application listen to client requests on port no 8081. You can specify any available port over here.

Run the above web server by writing node helloserver.js command in command prompt or terminal window and it will display message of node.js is running of port no. 8081.

The `http.createServer()` method includes request and response parameters which is supplied by Node.js. The request object can be used to get information about the current HTTP request e.g., url, request header, and data. The response object can be used to send a response for a current HTTP request.

```
var http = require('http'); // Import Node.js core module

var server = http.createServer(function (req, res) { //create web server
  if (req.url == '/') { //check the URL of the current request

    // set response header
    res.writeHead(200, { 'Content-Type': 'text/html' });

    // set response content
    res.write('<html><body><p>This is home Page.</p></body></html>');
    res.end();

  }
  else if (req.url == "/student") {

    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<html><body><p>This is student Page.</p></body></html>');
    res.end();
  }
});
```



```
}
else if (req.url == "/admin")
{
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('<html><body><p>This is admin Page.</p></body></html>');
  res.end();
}
else
  res.end('Invalid Request!');
});
server.listen(5000); //6 - listen for any incoming requests
console.log('Node.js web server at port 5000 is running.')
```

5.4.2 Reading Query String, Split Query String

In Node.js, functionality to aid in the accessing of URL query string parameters is built into the standard library. The built-in `url.parse` method takes care of most of the heavy lifting for us. Here is an example script using this handy function and an explanation on how it works:

```
const http = require('http');
const url = require('url');

http.createServer(function (req, res) {
  const queryObject = url.parse(req.url, true).query;
  console.log(queryObject);

  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('Feel free to add query parameters to the end of the url');
}).listen(8080);
```

To test this code run `node app.js` (`app.js` is name of the file) on the terminal and then go to your browser and type `http://localhost:8080/app.js?foo=bad&baz=foo` on the URL bar.

The key part of this whole script is this line: `const queryObject = url.parse(req.url, true).query;`. Let's take a look at things from the inside-out. First off, `req.url` will look like `/app.js?foo=bad&baz=foo`. This is the part that is in the URL bar of the browser. Next, it gets passed to `url.parse` which parses out the various elements of the URL (NOTE: the second parameter is a boolean stating whether the method should parse the query string, so we set it to `true`). Finally, we access the `.query` property, which returns us a nice, friendly JavaScript object with our query string data.

The `url.parse()` method returns an object which have many key value pairs one of which is the query object. Some other handy information returned by the method include `host`, `pathname`, `search` keys.



In the above code:

- `url.parse(req.url,true).query` returns `{ foo: 'bad', baz: 'foo' }`.
- `url.parse(req.url,true).host` returns `'localhost:8080'`.
- `url.parse(req.url,true).pathname` returns `'/app.js'`.
- `url.parse(req.url,true).search` returns `'?foo=bad&baz=foo'`.

Parsing with querystring

Another way to access query string parameters is parsing them using the `querystring` builtin Node.js module.

This method, however, must be passed just a querystring portion of a url. Passing it the whole url, like you did in the `url.parse` example, won't parse the querystrings.

```
const querystring = require('querystring');
const url = "http://example.com/index.html?code=string&key=12&id=false";
const qs = "code=string&key=12&id=false";
console.log(querystring.parse(qs));
// > { code: 'string', key: '12', id: 'false' }

console.log(querystring.parse(url));
// > { 'http://example.com/index.html?code': 'string', key: '12', id: 'false' }
```

5.5 File System Module

This module, as the name suggests, enables developers to work with the file system on its computers. The `require()` method is used to include the File System module. Some of the most common uses of this module is to create, read, update, delete, rename or read files. For example, the function `fs.readFile()` is used to read files on the computer.

5.5.1 Read files (`readFile()`)

The Node.js file system module (`fs` module) allows you to work with the files. The `fs.readFile()` is used to read from file.

There are two ways to read file

1. Read file Asynchronously:

```
var fs = require('fs');
fs.readFile('my_file.txt', (err, data) => {
  if (err){
    throw err;
  }else{
    console.log("Content of file is: " + data);
  }
});
```



2. Read file Synchronously:

```
var fs = require('fs');
var filename = 'my_file.txt';
var content = fs.readFileSync(filename);
console.log('Content of file is: ' + content);
```

5.5.2 Create Files (appendFile(), open (), writeFile())

There are a bunch of methods used for creating new files are: fs.appendFile (), fs.open (), fs.writeFile ().

The append method is used to, as the name suggests, append the given file. Also, if one particular file is appended but does not exist then a file of the same name will be created. The syntax of fs.appendFile () would look something like -

```
fs.appendFile('newfile.txt', 'Hello Konfinity!', function (err) {
  if (err) throw err;
  console.log('Done!');
});
```

The next method to discuss is the fs.open() method. This method takes a "flag" as the second argument. If the flag is "w", the specified file is opened for writing and if the called file does not exist then an empty file is created. The syntax of the fs.open() method will look like

```
fs.open('newfile2.txt', 'w', function (err, file) {
  if (err) throw err;
  console.log('Done!');
});
```

If you want to replace a particular file and its contents then the fs.writeFile() method is used. Also, if the file doesn't already exist, a new one will be created.

```
fs.writeFile('newfile3.txt', 'Hello Konfinity!', function (err) {
  if (err) throw err;
  console.log('Saved!');
});
```

Apart from these methods included in the file system module of Nodejs, there are a couple of other methods included too. Let's look at some of them and understand the concept in detail.

5.5.3 Update Files (appendFile(), writeFile())

The methods generally used to update files in a system are fs.appendFile() and fs.writeFile().

fs.appendFile() is a method that appends particular content at the end of the file mentioned in the code. The syntax for the method fs.appendFile() would look like -

```
fs.appendFile('newfile1.txt', ' This is my text.', function (err) {
  if (err) throw err;
  console.log('Updated!');
```



```
});
```

This code would append the text "This is my text." to the end of the file that goes by the name "newfile1.txt".

Another method is the `fs.writeFile()` one which replaces the file and content mentioned in the code. The syntax of the method is:

```
fs.writeFile('mynewfile3.txt', 'This is my text', function (err) {  
  if (err) throw err;  
  console.log('Replaced!');  
});
```

The code above replaces the content of the file "newfile3.txt": There are methods to delete files too.

5.5.4 Delete Files (`unlink()`)

The `fs.unlink()` method is used to delete a particular file with the File System module. The syntax of this method will look like -

```
fs.unlink('newfile2.txt', function (err) {  
  if (err) throw err;  
  console.log('File deleted!');  
});
```

The code basically deletes the file "newfile2.txt":

5.5.5 Rename Files (`rename()`)

In Nodejs, developers can also upload files to their computer. Apart from this, the file system module can also be used to rename files. The `fs.rename()` method is used to rename a file. This method renames the file mentioned in the code, the syntax for the same will look like -

```
fs.rename('vbp.txt', 'vbp1.txt', function (err) {  
  if (err) throw err;  
  console.log('File Renamed!');  
});
```

The code above, renames the file "vbp.txt" to "vbp1.txt":

Also keep in mind that all the methods mentioned above is prefixed with a variable `fs`. In these examples above, we took the variable to be 'fs', while executing your code, you would have to explicitly mention it in your code. The code will look something like - `var fs = require('fs');`